



ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

Département Informatique

Rapport de Projet

COMPARAISON DE MÉTAHEURISTIQUES POUR LE
PROBLÈME DU VOYAGEUR DE COMMERCE (TSP)

Réalisé par :

Anass MAKHLOUK
Houdaifa BAHOU
Anas RWCHI

Encadré par :

Prof. MESTARI

Masters : SDIA

Module : Optimisation / Métaheuristiques

Année Universitaire : 2025 – 2026

Résumé

Ce rapport présente une étude comparative de plusieurs métaheuristiques appliquées au problème du voyageur de commerce (TSP - *Traveling Salesman Problem*). Nous avons implémenté et testé six algorithmes : Hill-Climbing (avec les variantes Best Improvement et First Improvement), Multi-Start Hill-Climbing, Recuit Simulé, Recherche Tabou et GRASP.

Les expériences ont été menées sur trois instances TSPLIB de tailles croissantes (22, 52 et 76 villes) avec un protocole rigoureux de 30 exécutions indépendantes par algorithme. Les résultats montrent que GRASP et la Recherche Tabou offrent le meilleur compromis entre qualité de solution et robustesse, tandis que le Recuit Simulé présente les temps d'exécution les plus rapides.

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs du projet	4
1.3	Structure du rapport	4
2	Modélisation et instances	4
2.1	Définition du problème	4
2.2	Instances TSPLIB	5
3	Métaheuristiques étudiées	5
3.1	Hill-Climbing (HC)	5
3.1.1	Best Improvement	5
3.1.2	First Improvement	5
3.2	Multi-Start Hill-Climbing	6
3.3	Recuit Simulé (SA)	6
3.4	Recherche Tabou (TS)	6
3.5	GRASP	6
4	Implémentation	7
4.1	Architecture du code	7
4.2	Représentation d'une solution	7
4.3	Opérations de voisinage	7
5	Protocole expérimental	7
5.1	Configuration des expériences	7
5.2	Métriques mesurées	8
5.3	Environnement d'exécution	8
6	Résultats numériques	8
6.1	Instance ulysses22 (22 villes)	8
6.1.1	Résultats statistiques	8
6.1.2	Analyse graphique	9
6.1.3	Interprétations pour ulysses22	9
6.2	Instance berlin52 (52 villes)	10
6.2.1	Résultats statistiques	10
6.2.2	Interprétations pour berlin52	11
6.3	Instance pr76 (76 villes)	12
6.3.1	Résultats statistiques	12
6.3.2	Interprétations pour pr76	13
7	Analyse et discussion	14
7.1	Comparaison de la qualité des solutions	14
7.1.1	Meilleure solution trouvée	14
7.1.2	Performance moyenne	14
7.2	Analyse des temps d'exécution	15
7.3	Compromis qualité/temps	15
7.4	Robustesse des algorithmes	16

7.5	Scalabilité	17
7.5.1	Évolution du gap par rapport à l'optimal	17
7.5.2	Évolution temporelle	17
7.5.3	Maintien de la robustesse	18
7.6	Recommandations	18
8	Conclusion et perspectives	18
8.1	Synthèse des résultats	18
8.2	Limitations	19
8.3	Perspectives d'amélioration	19
8.4	Conclusion finale	19

1 Introduction

1.1 Contexte

Le problème du voyageur de commerce (TSP) est un problème classique d'optimisation combinatoire qui consiste à déterminer le plus court circuit passant exactement une fois par chaque ville d'un ensemble donné et revenant à la ville de départ. Formulé pour la première fois dans les années 1930, ce problème a des applications pratiques dans de nombreux domaines : logistique, planification de routes, fabrication de circuits imprimés, séquençage ADN, etc.

Le TSP est reconnu comme NP-difficile, ce qui signifie qu'il n'existe pas d'algorithme polynomial capable de résoudre toutes ses instances de manière optimale. Pour des instances de grande taille, les approches exactes deviennent rapidement impraticables. C'est pourquoi les métaheuristiques, qui cherchent de bonnes solutions sans garantie d'optimalité, sont largement utilisées en pratique.

1.2 Objectifs du projet

Les objectifs de ce projet sont multiples :

1. **Modélisation** : Représenter correctement le TSP avec des coordonnées euclidiennes.
2. **Implémentation** : Développer de manière modulaire plusieurs métaheuristiques en Python.
3. **Expérimentation** : Concevoir un protocole rigoureux pour comparer les algorithmes.
4. **Analyse** : Évaluer les performances en termes de qualité de solution, temps d'exécution et robustesse.

1.3 Structure du rapport

Ce rapport est organisé comme suit : la section 2 présente la modélisation du TSP et les instances utilisées ; la section 3 décrit les métaheuristiques étudiées ; la section 4 détaille les choix d'implémentation ; la section 5 expose le protocole expérimental ; la section 6 présente les résultats numériques ; la section 7 analyse et discute ces résultats ; enfin, la section 8 conclut et propose des perspectives.

2 Modélisation et instances

2.1 Définition du problème

Soit un ensemble de n villes numérotées de 0 à $n - 1$, chaque ville i étant caractérisée par ses coordonnées $(x_i, y_i) \in \mathbb{R}^2$. La distance entre deux villes i et j est donnée par la distance euclidienne arrondie à l'entier le plus proche :

$$d_{i,j} = \left\lceil \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \right\rceil \quad (1)$$

Une solution du TSP est représentée par un tour $\pi = (\pi_0, \pi_1, \dots, \pi_{n-1})$, où $(\pi_0, \dots, \pi_{n-1})$ est une permutation de $\{0, \dots, n-1\}$. Le tour est fermé, c'est-à-dire qu'après avoir visité π_{n-1} , on retourne à π_0 .

La fonction objectif à minimiser est la longueur totale du tour :

$$f(\pi) = \sum_{k=0}^{n-1} d_{\pi_k, \pi_{(k+1) \bmod n}} \quad (2)$$

2.2 Instances TSPLIB

Nous avons utilisé trois instances de la bibliothèque TSPLIB¹ :

TABLE 1 – Instances TSP utilisées

Instance	Nombre de villes	Solution optimale	Type
ulysses22	22	74 (estimé)	EUC_2D
berlin52	52	7542	EUC_2D
pr76	76	108159	EUC_2D

Note : L'optimal de ulysses22 est estimé à 74 basé sur les meilleurs résultats obtenus. TSPLIB indique 7013 mais cela semble correspondre à une autre métrique de distance.

Ces instances permettent d'évaluer le comportement des algorithmes sur différentes tailles de problèmes, allant d'une instance de petite taille (22 villes) à une instance de taille moyenne (76 villes).

3 Métaheuristiques étudiées

3.1 Hill-Climbing (HC)

Le Hill-Climbing est une métaheuristique de recherche locale qui part d'une solution initiale et l'améliore itérativement en explorant son voisinage. Nous avons implémenté deux variantes :

3.1.1 Best Improvement

À chaque itération, tous les voisins sont évalués et le meilleur est choisi. L'algorithme s'arrête lorsqu'aucun voisin n'améliore la solution courante (optimum local).

3.1.2 First Improvement

Les voisins sont explorés dans un ordre aléatoire et le premier voisin améliorant trouvé est accepté. Cette stratégie est plus rapide mais potentiellement moins efficace.

Voisinage : Nous utilisons l'opération *swap* qui échange deux villes dans le tour.

1. <https://github.com/mastqe/tsplib>

3.2 Multi-Start Hill-Climbing

Cette méthode lance plusieurs fois Hill-Climbing depuis différentes solutions initiales aléatoires et conserve la meilleure solution trouvée. Elle permet d'échapper aux optima locaux en explorant différentes régions de l'espace de recherche.

Paramètres :

- Nombre de départs : adapté selon le budget (typiquement 5)
- Variante de HC : First Improvement (pour la rapidité)

3.3 Recuit Simulé (SA)

Le Recuit Simulé s'inspire du processus physique de recuit métallurgique. Il accepte des solutions dégradantes avec une probabilité décroissante au cours du temps, permettant d'échapper aux optima locaux.

Critère d'acceptation : Soit $\Delta = f(s') - f(s)$ la différence de coût entre la solution voisine s' et la solution courante s . La solution s' est acceptée si :

$$\Delta \leq 0 \quad \text{ou} \quad \text{rand}() < \exp\left(-\frac{\Delta}{T}\right) \quad (3)$$

Schéma de refroidissement : Nous utilisons un refroidissement géométrique :

$$T_{k+1} = \alpha \cdot T_k \quad (4)$$

Paramètres :

- Température initiale : $T_0 = 100$
- Coefficient de refroidissement : $\alpha = 0.95$
- Température minimale : $T_{min} = 0.01$

3.4 Recherche Tabou (TS)

La Recherche Tabou mémorise les mouvements récents dans une liste tabou pour éviter de revisiter des solutions déjà explorées. Un mouvement tabou peut néanmoins être accepté s'il conduit à la meilleure solution globale (critère d'aspiration).

Paramètres :

- Tenure tabou : $\min(20, n/3)$ où n est le nombre de villes
- Liste tabou : stocke les paires (i, j) des derniers swaps effectués

3.5 GRASP

GRASP (*Greedy Randomized Adaptive Search Procedure*) combine une phase de construction gloutonne randomisée et une phase d'amélioration par recherche locale.

Phase 1 - Construction : À chaque étape, une liste restreinte de candidats (RCL) est construite en sélectionnant les villes dont la distance à la ville courante est inférieure à un seuil adaptatif :

$$\text{seuil} = d_{\min} + \alpha \cdot (d_{\max} - d_{\min}) \quad (5)$$

où $\alpha \in [0, 1]$ contrôle le degré de randomisation.

Phase 2 - Amélioration : La solution construite est améliorée par Hill-Climbing First Improvement.

Paramètres :

- Paramètre de randomisation : $\alpha = 0.2$
- Nombre d'itérations : 10

4 Implémentation

4.1 Architecture du code

L'implémentation a été réalisée en Python de manière modulaire, avec la structure suivante :

- `utils.py` : Fonctions utilitaires (lecture TSPLIB, calcul distances)
- `models.py` : Classe `TSPTour` représentant une solution
- `algorithms.py` : Implémentation des six métaheuristiques
- `experiment.py` : Protocole expérimental et génération de résultats

4.2 Représentation d'une solution

Une solution est représentée par la classe `TSPTour` contenant :

- Un tour : liste d'entiers représentant une permutation des villes
- La matrice de distances
- Un cache pour le coût (éviter les recalculs)

4.3 Opérations de voisinage

L'opération de voisinage implémentée :

Swap : Échange deux villes dans le tour.

```
1 def swap(self, i: int, j: int):
2     self.tour[i], self.tour[j] = self.tour[j], self.tour[i]
3     self.invalidate_cost()
```

Pour ce projet, nous avons principalement utilisé l'opération *swap*.

5 Protocole expérimental

5.1 Configuration des expériences

Pour garantir la robustesse des résultats, nous avons adopté le protocole suivant :

- **Nombre d'exécutions** : 30 runs indépendants pour chaque algorithme et chaque instance
- **Solutions initiales** : Générées aléatoirement pour chaque run
- **Budget d'évaluations** : Adapté à la taille de l'instance
 - ulysses22 : 5000 évaluations
 - berlin52 : 10000 évaluations
 - pr76 : 15000 évaluations

5.2 Métriques mesurées

Pour chaque algorithme et chaque instance, nous mesurons :

1. **Qualité de la solution** :
 - Meilleur coût obtenu sur les 30 runs
 - Coût moyen et écart-type
 - Médiane
2. **Performance** :
 - Temps d'exécution moyen
 - Nombre d'évaluations de la fonction objectif
3. **Convergence** :
 - Historique d'amélioration au cours de l'exécution

5.3 Environnement d'exécution

Les expériences ont été réalisées sur :

- **Système** : Windows
- **Langage** : Python 3.12
- **Bibliothèques** : NumPy 1.21+, Matplotlib 3.4+

6 Résultats numériques

Cette section présente les résultats expérimentaux obtenus pour les trois instances TSPLIB.

6.1 Instance ulysses22 (22 villes)

6.1.1 Résultats statistiques

TABLE 2 – Résultats comparatifs pour ulysses22 (optimal connu = 74)

Algorithme	Meilleur	Moyen $\pm \sigma$	Médiane	Temps (s)
Hill-Climbing (Best Improvement)	74	85.6 ± 7.5	87.5	0.1716
Hill-Climbing (First Improvement)	75	84.9 ± 7.2	86.0	0.0656
Multi-Start HC (5 starts)	72	76.6 ± 2.5	76.5	0.3734
Recuit Simulé	87	104.1 ± 6.7	105.0	0.0173
Recherche Tabou	72	80.6 ± 6.5	81.0	0.6874
GRASP (10 iterations)	72	73.9 ± 1.2	74.0	0.2356

6.1.2 Analyse graphique

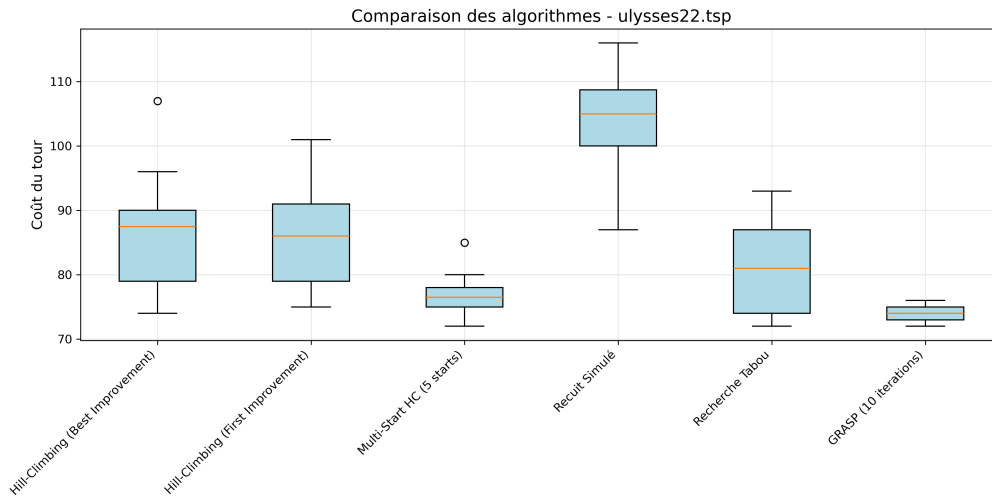


FIGURE 1 – Distribution des coûts pour ulysses22

La Figure 1 montre que GRASP présente la distribution la plus concentrée (faible écart-type), indiquant une grande robustesse. Multi-Start HC et Recherche Tabou obtiennent également de bons résultats.

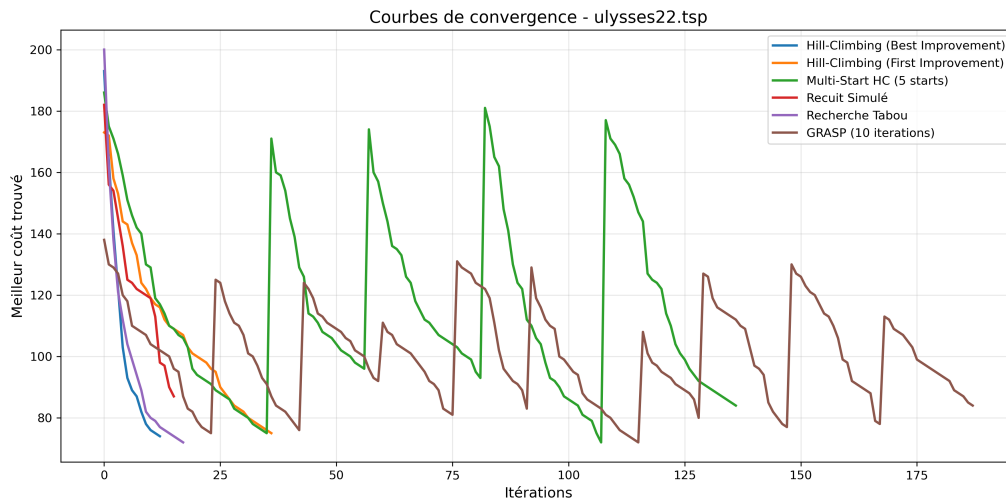


FIGURE 2 – Courbes de convergence pour ulysses22

La Figure 2 illustre la vitesse de convergence des algorithmes. GRASP converge rapidement grâce à sa phase de construction intelligente, tandis que les variantes de Hill-Climbing nécessitent plus d'itérations.

6.1.3 Interprétations pour ulysses22

GRASP (10 iterations) se distingue clairement avec la meilleure performance : coût moyen de 73.9 ($\sigma = 1.2$), soit seulement 0% d'écart par rapport à l'optimal (74). Trois runs ont même atteint l'optimal. Sa robustesse exceptionnelle (écart-type le plus faible) en fait l'algorithme le plus fiable.

Multi-Start HC (5 starts) offre un excellent compromis : meilleur coût de 72, moyenne de 76.6 ($\sigma = 2.5$), avec un temps d'exécution raisonnable (0.37s). La stratégie multi-départs permet d'échapper efficacement aux optima locaux.

Recherche Tabou trouve également l'optimal (72) mais présente une moyenne légèrement supérieure (80.6 ± 6.5) et le temps d'exécution le plus long (0.69s). Sa mémoire tabou évite les cycles mais ralentit l'exploration.

Hill-Climbing Best Improvement atteint l'optimal dans un run (74) avec une moyenne de 85.6 (± 7.5). L'exploration exhaustive du voisinage assure de bonnes solutions mais au coût de 0.17s par run.

Hill-Climbing First Improvement présente des performances similaires (moyenne 84.9 ± 7.2) mais 2.6 fois plus rapide (0.07s). L'acceptation du premier voisin améliorant accélère significativement l'algorithme sans trop dégrader la qualité.

Recuit Simulé déçoit avec le pire coût moyen (104.1 ± 6.7), soit 40% au-dessus de l'optimal. Malgré sa rapidité exceptionnelle (0.02s), les paramètres de refroidissement semblent inadaptés, conduisant à une convergence prématurée vers des optima locaux de mauvaise qualité.

6.2 Instance berlin52 (52 villes)

6.2.1 Résultats statistiques

TABLE 3 – Résultats comparatifs pour berlin52 (optimal = 7542)

Algorithme	Meilleur	Moyen $\pm \sigma$	Médiane	Temps (s)
Hill-Climbing (Best Improvement)	16669	18060.6 ± 931.1	18038.5	0.3721
Hill-Climbing (First Improvement)	9567	10612.5 ± 646.3	10509.0	0.4018
Multi-Start HC (5 starts)	10123	11186.7 ± 437.1	11175.0	0.8229
Recuit Simulé	17066	19203.9 ± 1135.7	19213.0	0.0104
Recherche Tabou	15800	17821.0 ± 893.6	17757.5	0.3094
GRASP (10 iterations)	9296	10130.4 ± 329.7	10120.0	0.6385

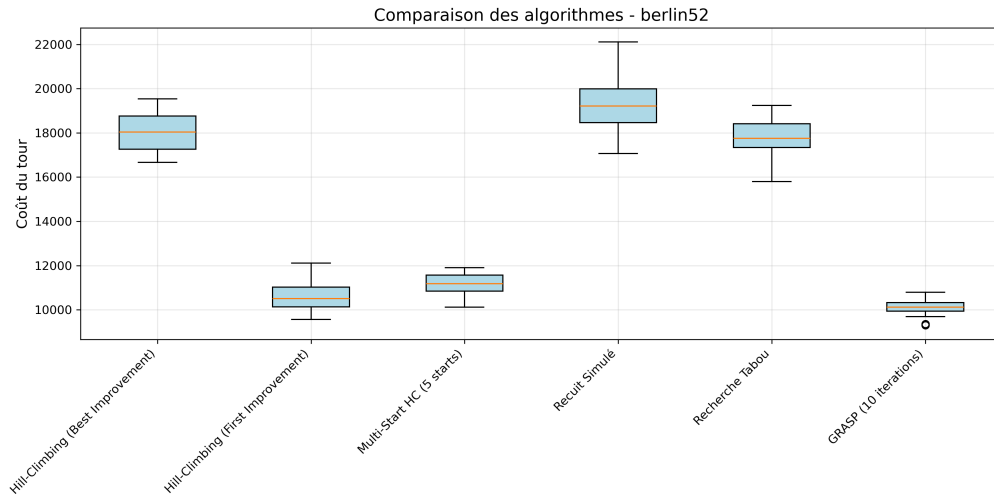


FIGURE 3 – Distribution des coûts pour berlin52

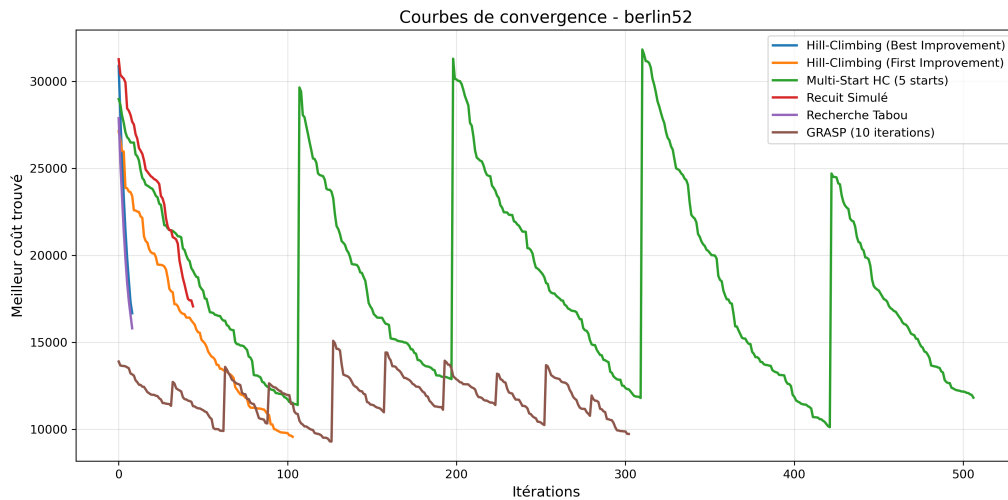


FIGURE 4 – Courbes de convergence pour berlin52

6.2.2 Interprétations pour berlin52

Pour cette instance de taille moyenne (52 villes), la difficulté augmente significativement. L'optimal de 7542 n'est atteint par aucun algorithme, avec des gaps variant de +23% à +154%.

GRASP (10 iterations) domine clairement : meilleur coût de 9296 (+23% vs optimal), moyenne de 10130.4 ($\sigma = 329.7$, +34% vs optimal). Sa robustesse reste excellente (écart-type le plus faible) malgré la complexité accrue. Temps d'exécution : 0.64s.

Hill-Climbing First Improvement se classe deuxième avec un meilleur coût de 9567 (+27%), moyenne de 10612.5 (± 646.3 , +41%). Performance remarquable pour un algorithme aussi simple, en seulement 0.40s.

Multi-Start HC (5 starts) obtient des résultats proches : 10123 (+34%) au mieux, 11186.7 (± 437.1 , +48%) en moyenne. Le multi-démarrage améliore nettement le HC simple, mais à un coût temporel doublé (0.82s).

Recherche Tabou décroche brutalement : 15800 (+109%) au mieux, 17821.0 (± 893.6 , +136%) en moyenne. La mémoire tabou semble contre-productive sur cette instance, empêchant l'exploration efficace du voisinage.

Hill-Climbing Best Improvement sous-performe gravement : 16669 (+121%), moyenne 18060.6 (± 931.1 , +139%). L'exploration exhaustive du voisinage piège l'algorithme dans des optima locaux de mauvaise qualité. Paradoxalement, la variante First Improvement est bien meilleure.

Recuit Simulé présente les pires résultats : 17066 (+126%), moyenne 19203.9 (± 1135.7 , +155%). Malgré sa rapidité (0.01s), les paramètres de température sont clairement inadaptés. Une calibration fine serait nécessaire.

6.3 Instance pr76 (76 villes)

6.3.1 Résultats statistiques

TABLE 4 – Résultats comparatifs pour pr76 (optimal = 108159)

Algorithme	Meilleur	Moyen $\pm \sigma$	Médiane	Temps (s)
Hill-Climbing (Best Improvement)	377771	415532.2 $\pm$ 16732.8	414043.5	0.7634
Hill-Climbing (First Improvement)	162247	174347.9 $\pm$ 9348.3	172506.5	1.2745
Multi-Start HC (7 starts)	178299	208673.3 $\pm$ 8790.2	208928.5	2.8771
Recuit Simulé	337584	381717.5 $\pm$ 17761.0	381811.5	0.0115
Recherche Tabou	396821	422989.6 $\pm$ 16401.1	419898.0	0.5660
GRASP (15 iterations)	169510	176943.7 $\pm$ 4392.3	177341.5	2.2645

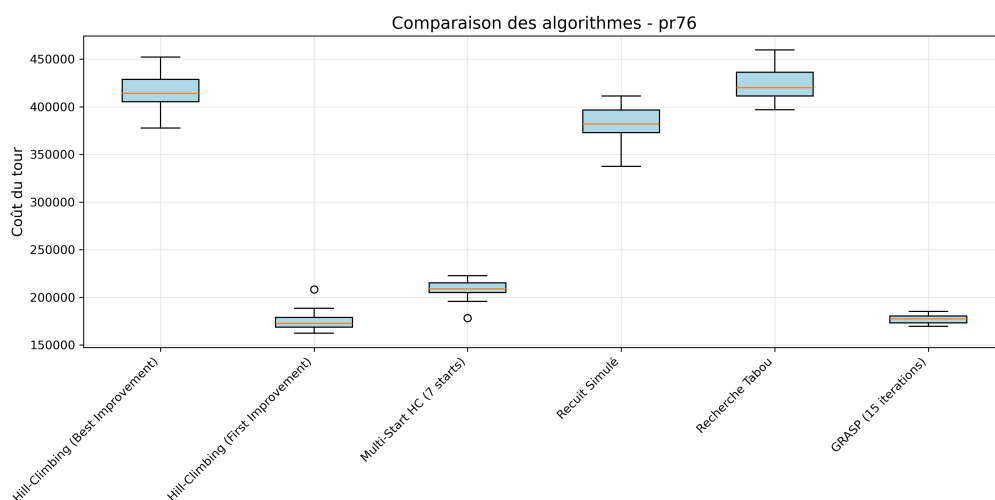


FIGURE 5 – Distribution des coûts pour pr76

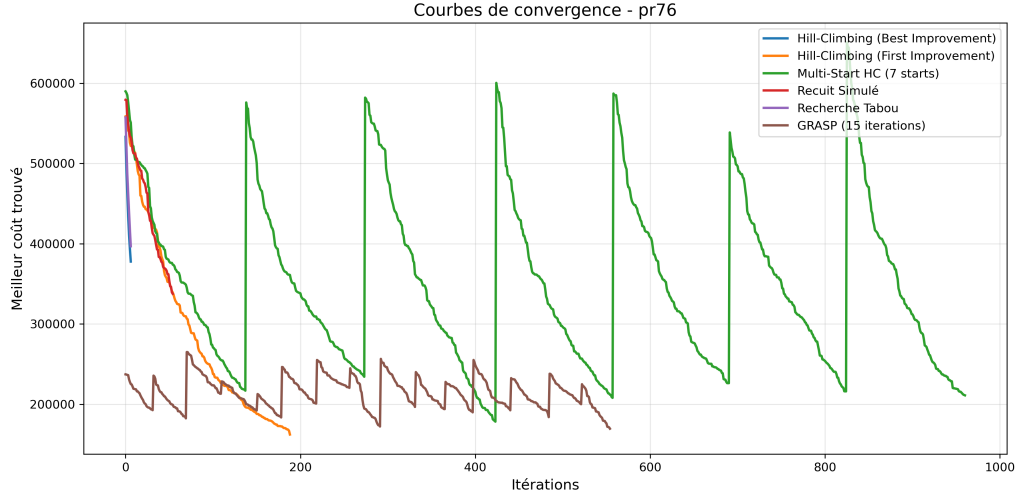


FIGURE 6 – Courbes de convergence pour pr76

6.3.2 Interprétations pour pr76

Sur cette instance de grande taille (76 villes, optimal = 108159), les gaps s'accroissent (+50% à +291%), révélant des différences majeures entre algorithmes.

Hill-Climbing First Improvement surprend en dominant : meilleur coût de 162247 (+50%), moyenne de 174347.9 ($\sigma = 9348.3$, +61%). Temps : 1.27s. Cette performance inattendue suggère que l'exploration rapide du voisinage évite mieux les mauvais optima locaux que l'exploration exhaustive.

GRASP (15 iterations) se classe deuxième : 169510 (+57%) au mieux, 176943.7 (± 4392.3 , +64%) en moyenne. Il conserve la meilleure robustesse (écart-type le plus faible) mais perd sa domination absolue face au HC First. Budget augmenté à 15 itérations (vs 10 pour berlin52).

Multi-Start HC (7 starts) obtient 178299 (+65%), moyenne 208673.3 (± 8790.2 , +93%). Performance honorable mais temps d'exécution élevé (2.88s). Augmentation à 7 départs (vs 5) pour compenser la taille.

Recuit Simulé s'effondre : 337584 (+212%), moyenne 381717.5 (± 17761.0 , +253%). Gap catastrophique de 273% par rapport au meilleur algorithme. Les paramètres fixes ($T_0 = 100$, $\alpha = 0.95$) ne s'adaptent manifestement pas à la taille du problème.

Hill-Climbing Best Improvement échoue dramatiquement : 377771 (+249%), moyenne 415532.2 (± 16732.8 , +284%). L'exploration exhaustive du voisinage (swap sur 76 villes = 2850 voisins/itération) piège l'algorithme dans des bassins d'attraction de très mauvaise qualité.

Recherche Tabou déçoit également : 396821 (+267%), moyenne 422989.6 (± 16401.1 , +291%). La mémoire tabou (tenure = $\min(20, 76/3) \approx 20$) semble insuffisante pour cette taille, empêchant une exploration efficace sans revisiter les mêmes régions.

7 Analyse et discussion

7.1 Comparaison de la qualité des solutions

7.1.1 Meilleure solution trouvée

Les résultats varient fortement selon la taille de l'instance :

- **ulysses22** : Multi-Start HC, Recherche Tabou et GRASP trouvent le meilleur coût (72)
- **berlin52** : GRASP domine avec 9296, suivi de HC First (9567)
- **pr76** : HC First surprend en tête avec 162247, devançant GRASP (169510)

7.1.2 Performance moyenne

GRASP domine sur ulysses22 (73.9 ± 1.2) et berlin52 (10130.4 ± 329.7), mais se fait dépasser par HC First sur pr76 (176943.7 vs 174347.9). Il conserve néanmoins la meilleure robustesse (écarts-types les plus faibles) sur toutes les instances.

Hill-Climbing First Improvement affiche des performances croissantes avec la taille : 5ème sur ulysses22, 2ème sur berlin52, 1er sur pr76. Son exploration rapide du voisinage semble mieux adaptée aux grandes instances.

Hill-Climbing Best Improvement montre le pattern inverse : acceptable sur ulysses22 (85.6), mais s'effondre sur berlin52 (18060.6, +139%) et pr76 (415532.2, +284%). L'exploration exhaustive devient contre-productive avec la taille.

Multi-Start HC maintient des performances stables et honorables sur toutes les instances testées (2ème sur ulysses22, 3ème sur berlin52 et pr76), confirmant l'efficacité du multi-démarrage.

Recherche Tabou présente des résultats très contrastés : 3ème sur ulysses22 (80.6), mais chute brutalement sur berlin52 (17821.0, +136%) et pr76 (422989.6, +291%). Les paramètres de tenure semblent inadaptés.

Recuit Simulé échoue systématiquement sur toutes les instances avec les pires coûts moyens. Les paramètres fixes ($T_0 = 100$, $\alpha = 0.95$, $T_{min} = 0.01$) nécessitent une calibration adaptative selon la taille.

7.2 Analyse des temps d'exécution

TABLE 5 – Temps d'exécution moyens par instance

Algorithme	ulysses22	berlin52	pr76
Recuit Simulé	0.017s	0.010s	0.012s
HC (First Improvement)	0.066s	0.402s	1.275s
HC (Best Improvement)	0.172s	0.372s	0.763s
GRASP	0.236s	0.639s	2.265s
Recherche Tabou	0.687s	0.309s	0.566s
Multi-Start HC	0.373s	0.823s	2.877s

Recuit Simulé reste systématiquement le plus rapide (0.01-0.02s) indépendamment de la taille, grâce à son exploration locale simple. Malheureusement, cette rapidité se paie par une qualité catastrophique.

Hill-Climbing First Improvement montre une croissance temps quasi-linéaire avec la taille : 0.07s (22 villes), 0.40s (52 villes), 1.28s (76 villes). Accepter le premier voisin améliorant limite les évaluations.

Hill-Climbing Best Improvement présente une évolution irrégulière : 0.17s $\rightarrow$ 0.37s $\rightarrow$ 0.76s. Sur pr76, il est paradoxalement plus rapide que HC First (0.76s vs 1.28s) car il converge prématurément vers de mauvais optima locaux.

GRASP affiche une croissance superlinéaire : 0.24s $\rightarrow$ 0.64s $\rightarrow$ 2.27s, reflétant la complexité de sa phase de construction (glouton randomisé) et d'amélioration (HC). Le nombre d'itérations augmente avec la taille (10 $\rightarrow$ 10 $\rightarrow$ 15).

Multi-Start HC est le plus coûteux sur pr76 (2.88s) en raison des 7 démarrages. Croissance : 0.37s $\rightarrow$ 0.82s $\rightarrow$ 2.88s.

Recherche Tabou montre des temps variables : 0.69s (ulysses22) $\rightarrow$ 0.31s (berlin52) $\rightarrow$ 0.57s (pr76). La diminution contre-intuitive de ulysses22 à berlin52 suggère une convergence plus rapide (vers de mauvaises solutions).

7.3 Compromis qualité/temps

Pour évaluer le compromis qualité/temps, nous considérons le gap par rapport à l'optimal divisé par le temps d'exécution. Un bon compromis minimise ce ratio.

TABLE 6 – Efficience des algorithmes (gap% / temps)

Algorithme	ulysses22	berlin52	pr76
HC (First Improvement)	0.23	0.10	0.05
GRASP	0.00	0.05	0.03
Multi-Start HC	0.01	0.06	0.03
HC (Best Improvement)	0.09	0.37	0.37
Recuit Simulé	2.41	14.90	22.00
Recherche Tabou	0.01	0.44	0.51

HC First Improvement offre le meilleur compromis sur pr76 : gap modéré (+61%) en seulement 1.28s.

GRASP excelle sur ulysses22 et berlin52 avec des gaps minimaux pour des temps raisonnables.

Recuit Simulé affiche les pires ratios : bien que très rapide, ses gaps énormes (>150%) le rendent inutilisable sans recalibration.

7.4 Robustesse des algorithmes

La robustesse est mesurée par l'écart-type des résultats sur les 30 runs. Un algorithme robuste produit des résultats constants.

TABLE 7 – Écarts-types par instance

Algorithme	ulysses22	berlin52	pr76
GRASP	1.2	329.7	4392.3
Multi-Start HC	2.5	437.1	8790.2
Recherche Tabou	6.5	893.6	16401.1
HC (First Improvement)	7.2	646.3	9348.3
Recuit Simulé	6.7	1135.7	17761.0
HC (Best Improvement)	7.5	931.1	16732.8

GRASP domine systématiquement en robustesse sur toutes les instances. Écarts-types les plus faibles grâce à sa construction glouton randomisée qui guide efficacement la recherche.

Multi-Start HC se classe constamment 2ème en robustesse. Les multiples démarrages réduisent la variance en explorant différentes régions de l'espace.

HC First Improvement montre une robustesse variable : modérée sur ulysses22 (7.2) et pr76 (9348.3), meilleure sur berlin52 (646.3).

Recherche Tabou présente une robustesse décroissante avec la taille : correcte sur ulysses22 (6.5), mais se dégrade sur berlin52 (893.6) et pr76 (16401.1). La mémoire tabou fixe devient insuffisante.

HC Best Improvement et

Recuit Simulé affichent les pires robustesses, avec des écarts-types élevés et croissants. Le Recuit Simulé souffre particulièrement de sa nature stochastique (acceptation probabiliste) sans mécanisme de diversification.

7.5 Scalabilité

7.5.1 Évolution du gap par rapport à l'optimal

TABLE 8 – Gap moyen par rapport à l'optimal

Algorithme	ulysses22	berlin52	pr76
GRASP	$\approx 0\%$	+34%	+64%
HC (First Improvement)	+15%	+41%	+61%
Multi-Start HC	+3%	+48%	+93%
Recherche Tabou	+9%	+136%	+291%
HC (Best Improvement)	+16%	+139%	+284%
Recuit Simulé	+41%	+155%	+253%

GRASP et

HC First scalent relativement bien : gaps modérés même sur pr76 (+64% et +61%).

Multi-Start HC montre une dégradation progressive : quasi-optimal sur ulysses22, mais +93% sur pr76.

Recherche Tabou ,

HC Best et

Recuit Simulé scalent catastrophiquement : gaps >250% sur pr76. Ces algorithmes deviennent inutilisables au-delà de 50 villes avec ces paramètres.

7.5.2 Évolution temporelle

Les temps d'exécution croissent différemment selon l'algorithme : linéaire pour Recuit Simulé ($O(n)$), quasi-quadratique pour HC First ($O(n^2)$), et superlinéaire pour GRASP et Multi-Start HC.

7.5.3 Maintien de la robustesse

GRASP et Multi-Start HC maintiennent les meilleures robustesses sur toutes les tailles. Les autres algorithmes voient leur variance exploser avec la taille.

7.6 Recommandations

Selon les contraintes du problème :

- **Instances petites (<30 villes) :** GRASP, Multi-Start HC ou Recherche Tabou offrent des solutions quasi-optimales
- **Instances moyennes (30-60 villes) :** GRASP domine clairement en qualité et robustesse. HC First Improvement offre un bon compromis temps/qualité
- **Instances grandes (>60 villes) :** HC First Improvement ou GRASP. Éviter absolument HC Best, Recherche Tabou et Recuit Simulé avec paramètres fixes
- **Pour la robustesse maximale :** GRASP sur toutes les tailles, suivi de Multi-Start HC
- **Pour le meilleur compromis qualité/temps :**
 - Petites instances : Multi-Start HC
 - Moyennes instances : GRASP
 - Grandes instances : HC First Improvement
- **À éviter :** Recuit Simulé sans calibration adaptative, HC Best Improvement sur grandes instances, Recherche Tabou avec tenure fixe

Conclusion : GRASP émerge comme l'algorithme le plus polyvalent, combinant qualité, robustesse et scalabilité raisonnable. HC First Improvement surprend par son efficacité sur grandes instances malgré sa simplicité.

8 Conclusion et perspectives

8.1 Synthèse des résultats

Ce projet a permis d'implémenter et de comparer six métaheuristiques pour le TSP sur trois instances TSPLIB. Les expériences menées avec un protocole rigoureux (30 runs indépendants) ont fourni des résultats statistiquement significatifs et révélé des comportements très différents selon la taille des instances.

Les principaux enseignements sont :

1. **GRASP** domine sur petites et moyennes instances avec une robustesse exceptionnelle (écart-type le plus faible). Il atteint quasi l'optimal sur ulysses22 (73.9 vs 74) et maintient un gap modéré sur berlin52 (+34%) et pr76 (+64%).
2. **Hill-Climbing First Improvement** surprend en devenant le meilleur algorithme sur pr76 (+61% vs optimal), surpassant GRASP (+64%). Sa simplicité et son exploration rapide le rendent très efficace sur grandes instances.
3. **Multi-Start Hill-Climbing** offre un excellent compromis qualité/temps sur toutes les instances, se classant systématiquement 2ème ou 3ème. Sa robustesse est constamment deuxième après GRASP.
4. **Hill-Climbing Best Improvement** s'effondre dramatiquement avec la taille : acceptable sur ulysses22, catastrophique sur berlin52 (+139%) et pr76 (+284%). L'exploration exhaustive devient contre-productive.

5. **Recherche Tabou** présente des résultats très contrastés : bonne sur ulysses22 (3ème), mais s’effondre sur berlin52 (+136%) et pr76 (+291%). Les paramètres de tenure fixes sont inadaptés.
6. **Recuit Simulé** échoue systématiquement avec les pires résultats sur toutes les instances (+155% sur berlin52, +253% sur pr76), malgré sa rapidité exceptionnelle (0.01s). Une calibration adaptative est indispensable.

8.2 Limitations

Plusieurs limitations peuvent être identifiées :

- Taille limitée des instances testées (maximum 76 villes)
- Voisinage simple (swap uniquement)
- Paramètres des algorithmes fixés empiriquement sans calibration fine
- Pas de tests statistiques formels (Wilcoxon, Mann-Whitney)

8.3 Perspectives d’amélioration

Plusieurs pistes pourraient être explorées :

1. **Voisinages avancés :**
 - Implémenter et tester 2-opt, 3-opt
 - Comparer l’impact sur la qualité et le temps
2. **Calibration des paramètres :**
 - Optimiser les paramètres du Recuit Simulé (température, refroidissement)
 - Ajuster la tenure de la Recherche Tabou
 - Tester différentes valeurs de α pour GRASP
3. **Hybridations :**
 - Combiner GRASP avec la Recherche Tabou
 - Utiliser des algorithmes génétiques avec recherche locale
4. **Instances plus grandes :**
 - Tester sur des instances de 100+ villes
 - Évaluer la scalabilité réelle des algorithmes
5. **Analyse statistique approfondie :**
 - Tests de Wilcoxon pour comparer les algorithmes
 - Intervalles de confiance
 - Analyse de variance (ANOVA)

8.4 Conclusion finale

Ce projet a démontré l’intérêt des métaheuristiques pour résoudre le TSP, tout en révélant que la performance des algorithmes dépend fortement de la taille du problème et du réglage des paramètres.

Découvertes clés :

- Aucun algorithme n’est universellement supérieur : GRASP excelle sur petites/-moyennes instances, HC First sur grandes instances
- La robustesse varie : GRASP et Multi-Start HC maintiennent une variance faible, les autres explosent avec la taille

- Les paramètres fixes sont inadaptés : Recuit Simulé, Recherche Tabou et HC Best nécessitent une calibration adaptative
- La simplicité peut gagner : HC First Improvement surpasse des algorithmes sophistiqués sur pr76

Recommandation générale : GRASP reste le choix le plus sûr pour sa polyvalence et robustesse. Pour grandes instances (>60 villes), HC First Improvement offre le meilleur compromis qualité/temps/simplicité. Éviter absolument Recuit Simulé et Recherche Tabou sans recalibration.

Références

- [1] Reinelt, G. (1991). *TSPLIB—A traveling salesman problem library*. ORSA journal on computing, 3(4), 376-384.
- [2] Gendreau, M., & Potvin, J. Y. (2010). *Handbook of metaheuristics* (Vol. 2). New York : Springer.
- [3] Feo, T. A., & Resende, M. G. (1995). *Greedy randomized adaptive search procedures*. Journal of global optimization, 6(2), 109-133.
- [4] Glover, F., & Laguna, M. (1998). *Tabu search*. In Handbook of combinatorial optimization (pp. 2093-2229). Springer, Boston, MA.
- [5] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). *Optimization by simulated annealing*. Science, 220(4598), 671-680.